

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON

CURSO 3: CREACIÓN Y DESARROLLO DE AGENTES DE IA (NIVEL 3 (AVANZADO))

Módulo 02: Herramientas, Memoria y RAG

«La Caja de Herramientas y el Bibliotecario con Memoria»

Dotando de superpoderes a los LLMs: Invocación de funciones (Tool Use / Function Calling), memoria a corto/largo plazo y RAG (Retrieval-Augmented Generation) con embeddings y bases de datos vectoriales.

Instructor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Nivel del Curso: Avanzado | **Python:** 3.10+ | **Licencia:** MIT

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre **wisrovi SUITE** en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Estructura Pedagógica de Cada Guía

Cada documento de esta serie está diseñado rigurosamente bajo el estándar de ingeniería de software profesional:

- **Fundamento Conceptual y Metáfora Intuitiva:** Anclaje visual del concepto abstracto a la realidad.
- **Diagrama de Flujo y Arquitectura Mental:** Representación visual de cómo fluye la información.
- **Código de Nivel Profesional:** Sintaxis limpia, comentarios línea a línea y tipado estático (PEP 484).
- **Patrones y Gotchas:** Errores comunes que cometen los desarrolladores y cómo evitarlos.

Índice General de la Sesión

Sección	Contenido y Enfoque Temático	Página
Capítulo 1	Fundamentos y Metáfora Conceptual: Capacidades Aumentadas: Tool Calling y RAG	Pág. 4
Capítulo 2	Diagrama de Flujo y Arquitectura: Arquitectura de un Sistema RAG (Retrieval-Augmented Generation)	Pág. 5
Capítulo 3	Implementación Práctica en Python: Definición y Ejecución de Herramientas (Tool Calling)	Pág. 6
Capítulo 4	Patrones Avanzados, Gotchas y Debugging: Gotchas en Tool Calling y RAG	Pág. 7
Cierre	Conclusiones, Notas del Mentor y Agradecimiento	Pág. 8
Anexos	Bibliografía Oficial y Enlaces de Profundización	Pág. 9

Objetivos de Aprendizaje (Competencias Clave)



Competencia Conceptual

Comprender cómo un modelo utiliza herramientas externas para ejecutar código y cómo RAG elimina alucinaciones inyectando contexto verificado.



Competencia Práctica

Implementar un pipeline RAG con búsqueda por similitud de coseno y definir herramientas Python ejecutables por el modelo.

Requisitos Previos y Entorno Recomendado

Para aprovechar al máximo esta sesión se recomienda tener instalado Python 3.10 o superior, Visual Studio Code con la extensión oficial de Python configurada, y haber completado las lecturas y ejercicios de las sesiones anteriores de la ruta formativa.

1. Capacidades Aumentadas: Tool Calling y RAG

Un LLM aislado solo puede generar texto; cuando le otorgas herramientas y memoria, se transforma en un agente inteligente capaz de interactuar con el mundo.

☀ **Metáfora Central: La Caja de Herramientas y el Bibliotecario con Memoria**

El LLM es como un consultor brillante pero encerrado en una habitación: Tool Calling es darle un teléfono para llamar a APIs o consultar bases de datos. RAG es ponerle al lado a un bibliotecario que busca en segundos los documentos exactos de tu empresa y se los pasa antes de que responda.

Principios Teóricos y Modelo Mental

Tool Calling: El LLM decide qué función invocar y genera los argumentos exactos; tu código ejecuta la función y devuelve el resultado al LLM.

Embeddings Vectoriales: Representaciones numéricas de significado semántico; textos con significados similares tienen alta similitud de coseno en el espacio vectorial.

⚡ **Regla de Oro en Python**

RAG resuelve el problema del conocimiento desactualizado y las alucinaciones sin necesidad de reentrenar el modelo fundacional.

2. Arquitectura de un Sistema RAG (Retrieval-Augmented Generation)

Flujo de ingestión, búsqueda semántica y generación contextualizada con documentos privados.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Los documentos se dividen en fragmentos (chunks) y se vectorizan con un modelo de embeddings.	Vectores en Vector DB
2. Evaluación	El usuario hace una pregunta; se vectoriza la consulta del usuario.	Query vector
3. Transformación	Se realiza búsqueda de similitud (k-Nearest Neighbors / Cosine Similarity) para extraer los chunks más relevantes.	Contexto recuperado
4. Retorno / Salida	Se ensambla el prompt enriquecido [Contexto + Pregunta] y el LLM formula la respuesta final respaldada en hechos.	Respuesta precisa y sin alucinación

Visualización Mental

RAG es el examen a libro abierto del LLM: en lugar de memorizar todo, le das el párrafo exacto donde está la respuesta.

3. Definición y Ejecución de Herramientas (Tool Calling)

Registro dinámico de funciones Python para ser ejecutadas autónomamente por el modelo:

main.py (Python 3.10+)

UTF-8 • PEP 8 Compliant

```
import json

# 1. Definición de la Herramienta en Python puro
def consultar_clima(ciudad: str) -> str:
    """Consulta la temperatura actual de una ciudad."""
    datos = {"Madrid": "24°C Despejado", "Bogotá": "18°C Lluvioso"}
    return datos.get(ciudad, "20°C Clima templado")

# 2. Registro de herramientas disponibles para el agente
HERRAMIENTAS_DISPONIBLES = {"consultar_clima": consultar_clima}

# 3. Simulación de orden de Tool Call emitida por el LLM
llamada_modelo = {
    "funcion": "consultar_clima",
    "argumentos": {"ciudad": "Madrid"}
}

# 4. Despachador de ejecución segura
nombre_fn = llamada_modelo["funcion"]
args = llamada_modelo["argumentos"]
resultado_tool = HERRAMIENTAS_DISPONIBLES[nombre_fn](**args)

print(f"Resultado de la herramienta para el LLM: {resultado_tool}")
```

Análisis del Código Fuente

Mecanismo de despacho dinámico mediante desempaqueado de argumentos (**kwargs) para conectar funciones locales con el LLM.

4. Gotchas en Tool Calling y RAG

Errores clásicos al implementar memorias y herramientas:

⚠ Gotcha Frecuente (Trampa de Principiante)

Fragmentar documentos en chunks demasiado grandes (que diluyen la relevancia semántica) o demasiado pequeños (que pierden contexto).

Patrón Recomendado vs Antipatrón

✗ Antipatrón / Mal Código

Chunks de 5000 tokens: el embedding pierde especificidad semántica

✓ Patrón Pythonic / Correcto

Chunks de 300-500 tokens con 50 tokens de solapamiento (overlap)

🛡 Consejo de Resiliencia en Producción

Incluye siempre docstrings claros y detallados en tus funciones Python; los LLMs leen esos docstrings para saber cuándo invocar la herramienta.

5. Resumen Ejecutivo y Conclusiones

Domina los dos pilares que transforman un modelo de lenguaje en un sistema interactivo: Tool Calling y RAG.



Logro Alcanzado en esta Clase

Capacidad para conectar modelos de IA con APIs externas, bases de datos vectoriales y fuentes documentales.

Notas del Instructor y Siguientes Pasos

En el siguiente módulo integraremos estos componentes en Agentes Autónomos con el ciclo de razonamiento ReAct.



Mensaje de Agradecimiento

Muchas gracias por tu entusiasmo, disciplina y dedicación al participar en este programa formativo. La programación es un superpoder que transforma vidas cuando se ejerce con constancia y curiosidad. ¡Nos vemos en la próxima sesión para seguir construyendo juntos!

«El código más elegante es aquel que cualquiera puede entender y nadie necesita reescribir.»

6. Fuentes Bibliográficas y Recursos de Estudio

Para profundizar y consolidar los conocimientos adquiridos en esta clase, consulta las siguientes referencias:

Recurso / Fuente	Descripción Temática	Enlace Oficial
Documentación Oficial de Python	Referencia canónica del lenguaje y librería estándar	docs.python.org/3/
PEP 8 - Style Guide for Python	Guía oficial de estilo, formato e indentación	peps.python.org
Real Python Tutorials	Artículos técnicos y patrones de desarrollo moderno	realpython.com
Python Type Checking (PEP 484)	Anotaciones de tipo y análisis estático en Python	docs.python.org/typing
Suite Open Source wisrovi	Paquetes Python para orquestación y alto rendimiento	github.com/wisrovi

Canales de Consulta y Comunidad

Si encuentras dificultades al replicar el código o tienes dudas sobre la arquitectura de los ejercicios, no dudes en abrir un Issue en el repositorio oficial del curso en GitHub o participar activamente en nuestras sesiones grupales de mentoría.



Desafío de Autoestudio Recomendado

Crea una herramienta que consulte una base de datos SQLite y permita a un LLM responder preguntas sobre inventarios.